G6_NC_OG_09-02

Mier - Teo - Estimaciones en ambientes ágiles

---

1. Gestión de productos de SW

- El primer parcial: la parte teórica ya está.
- Parte práctica: User Stories completa (todas sus partes). Respetar el formato de la US.
- Estimaciones de esa US y MVP.

---

2. Estimaciones

- Contiene incertidumbre: nos falta información de la gente que va a trabajar, el nivel de expertiz de esa gente.
- **Se confunde el objetivo que quiero lograr CON LA ESTIMACIÓN.**
- La estimación no es la realidad — está asociada a una **probabilidad**.

2.1 NoEstimate (#noestimate)

- Un equipo puede no estimar.
- Filosofía `#noestimate`.
- **Desperdicio:** algo es desperdicio cuando NO GENERA VALOR.
- En contraposición de la estimación, el NoEstimate dice que es un desperdicio la estimación porque la probabilidad asociada de accuracy es muy baja.
- Si no se estima hay que compensar con otra cosa.

> Las estimaciones en ambientes ágiles son diferentes a las estimaciones en ambientes tradicionales.

---

3. Agilismo y Lean

- "Uno es uno y sus circunstancias." No aislarse del contexto. NO HAY QUE SER DOGMÁTICO. Hay que ver en qué contexto y situación está en este momento, para actuar de acuerdo al contexto.

- Nadie hace SW como fin en sí mismo, siempre trabajamos para alguien. Es la **meta de negocio**.

3.1 Objetividad

- Con la estimación: no mezclar con certeza.
- Estimar es predecir una situación que va a ocurrir a futuro. Va a depender de lo que voy a estimar.
- **Compromiso** es lo que ESPERAMOS LOGRAR CON LA PLANIFICACIÓN, basándose en esa estimación.

El agilismo apunta a que el cliente forme parte del equipo — toman decisiones respecto a las cuestiones técnicas. Es más fácil manejar los imprevistos.

---

4. Subestimar y Sobreestimar

4.1 Subestimar

- Cuando el equipo está apurado y estima mal, recorta el tiempo de estimación y la estimación resulta mal.
- "Hagamoslo mal y después lo mejoramos": ese "después lo mejoramos" NO LLEGA NUNCA. Nunca hay refactoring.
- Queremos el código rápido y sucio — consecuencia: altas tasas de defectos.
- Cuando hay que hacer lo que haga falta para que funcione, se hace apurado y mal, y termina teniendo consecuencias.

4.2 Sobreestimar

- Decir que necesito más tiempo que el que en verdad voy a usar.

---

5. El Cono de la Incertidumbre

- Las estimaciones se piden demasiado pronto. El cliente las pide al principio, y ni siquiera tenés proyecto.
- **PRE estimaciones:** estás trabajando a riesgo.

---

6. Anatomía de una Buena Estimación

- Expresada en **rangos**. Nunca una fecha y hora fija. La precisión es cara — tenés una altísima probabilidad de no pegarle. "Entre tal y tal". Los rangos de oscilación tienen que ser cosas razonables.
- Incluye **porcentaje de probabilidad** específico.
- Totalmente separada del objetivo del negocio.

> Si vos decís un número, el cliente NO SE LO OLVIDA MÁS. Por eso hay que poner un rango para mover de +/- el 20%.

- Habilita decisiones de control.
- La estimación es el plano. El control de proyecto es la construcción.

---

7. Estimación Relativa

- Las personas no saben estimar en términos absolutos.
- Somos buenos comparando cosas.
- Comparar es generalmente más rápido.
- Se obtiene una mejor dinámica grupal y pensamiento de equipo más que individual.
- Se emplea mejor el tiempo de análisis de las stories.

La unidad de medida para estimar las historias de usuario se llama **Story Points (SP)**. El crecimiento en SW es exponencial — usamos estimaciones relativas.

> ¿Qué vamos a estimar de la US? — EL TAMAÑO.

Hacemos énfasis en que antes de pensar en **CUÁNTO Y CUÁNDO** tenemos que resolver, hay que pensar **QUÉ** es lo que vamos a hacer. Lo otro se va a derivar de lo que tenemos que hacer.

7.1 Tamaño != Esfuerzo

- **Tamaño:** refiere al producto que voy a construir.
- **Esfuerzo:** mide el trabajo necesario, en horas-personas lineales (una única tarea a la vez, una persona).
  - Poca gente estima esfuerzo. Se estima por tiempo y de manera muy optimista.
  - Se suele contestar desde el lugar equivocado.

El cuándo es una consecuencia. El cuándo y el cómo se resuelve después — PRIMERO VA EL QUÉ.

7.2 ¿Quién estima?

El que va a hacer el trabajo debe estimar — a diferencia del líder de proyecto.

- El líder de proyecto estima pero no sabe el esfuerzo que va a llevar y tampoco conoce a las personas.
- El agilismo dice que TIENE QUE ESTIMAR EL QUE HACE EL TRABAJO.
- Equipos autogestionados, empoderados para tomar decisiones.
- En lo tradicional, la responsabilidad está en el jefe de proyecto.

7.3 Técnicas de estimación

- **Talle de remera:** se recomienda para estimar los items en el product backlog.

7.4 Story Point

Es una unidad de medida específica (del equipo) de complejidad, riesgo y esfuerzo — es lo que "el kilo" a la unidad de nuestro sistema de medición de peso.

- Complejidad
- Esfuerzo: del que va a hacer el trabajo
- Incertidumbre

Bajar la incertidumbre porque más le vamos a pegar cuando estemos estimando la complejidad y el esfuerzo.

7.5 Velocidad (Velocity)

- Velocidad / Velocity es una **métrica** del progreso de un equipo.
- Se calcula sumando el número de story points (asignados a cada user story) que el equipo completa durante la iteración.
- La cantidad de puntos de historia que el Product Owner me aceptó al final de la iteración.
- La velocidad **no se estima** — se calcula. Se calcula sumando y multiplicando.
- Al momento que presentás el producto, se lo presentás al PO.

---

8. MVP, MMP y MLP

8.1 MVP (Minimum Viable Product)

- Voy a crear un producto que alguien quiere consumir? Pienso en un producto que sirve para alguien.
- No es necesariamente un producto que vayas a vender.
- La clave es que lo quieras usar, no solo escalarlo.

- **ÍNDICE DE RETENCIÓN:** determina si un producto sobrevive o no.
- **Validar una hipótesis.**
- Vamos a construir el producto. Vamos a conseguir inversores o no.
- Salir al mercado con una versión de producto limitada en funcionalidad pero con la misión de ser viable: factibilidad.

8.2 MMP (Minimal Marketable Product)

- Si tiene que ser SW funcionando: ¿cuál es la cantidad mínima de producto que puedo poner en producción?
- Por esto elegís ciclo de vida de SW iterativo incremental.

8.3 MLP (Minimal Lovable Product)

- Una sola oportunidad para que el usuario disfrute un producto — experiencia de usuario.
- No alcanza la funcionalidad sino de que sea "yo feliz usando el producto".
- No vas a tener otra primera oportunidad.
- Tener en cuenta desde el primer momento los RNF (requisitos no funcionales), porque se cree que tenés posibilidades de incorporarlo después.